

课程笔记

CS224N Lecture 12: Efficient Neural Network Training

整理：AI Course Notes Contributors；授课：Chris Manning

2024 年 5 月

课程版本：2024 年中文讲义整理版

笔记来源：AI Course Notes Contributors

本版处理：移除课件截图和对应图注，不包含 Stanford 原始课件、图片或视频。

授权：笔记依 CC BY-NC-SA 4.0 分享；完整许可与改动记录见下一页。

来源与许可

本中文笔记由 AI Course Notes Contributors 整理，课程版本为 2024；源稿：<https://github.com/hqhq1025/ai-course-notes/tree/717e2df65c3d2eee593c171156a30d4f8f6812b9/cs224n/lecture12>。

笔记内容依 CC BY-NC-SA 4.0 分享：<https://creativecommons.org/licenses/by-nc-sa/4.0/>。

本副本的改动：移除 Stanford 原始课件截图与依赖这些截图的图注，移除课程视频封面/链接，并清理 TeX 源稿中的行尾空白后重新编译为可独立阅读的 PDF；同时加入本来源与许可说明。完整许可文本随本文件夹提供。Stanford 课件、课件图片和视频不包含在该授权内；请从对应年份 Stanford 课程页访问原始课件。

目录

1 引言：高效训练神经网络的重要性	3
1.1 本章小结	3
2 浮点数表示基础	3
2.1 FP32：标准单精度浮点数	3
2.2 FP16：半精度浮点数	4
2.3 本章小结	4
3 混合精度训练	4
3.1 初步方案：FP32 主权重 + FP16 计算	4
3.2 Loss Scaling：解决梯度下溢	5
3.3 BFloat16：更优雅的解决方案	6
3.4 混合精度训练的实际效果	6
3.5 本章小结	7
4 多 GPU 训练：DDP 与 ZeRO	7
4.1 GPU 显存占用分析	7
4.2 分布式数据并行 (DDP)	7
4.3 DDP 的内存问题	8
4.4 ZeRO Stage 1：分片优化器状态	8
4.5 ZeRO Stage 2：分片梯度	9
4.6 ZeRO Stage 3 / FSDP：分片模型参数	9
4.7 GPU 显存中被忽视的部分：模型激活值	10
4.8 本章小结	10
5 参数高效微调：LoRA	10
5.1 为什么需要参数高效微调	10
5.2 全量微调 vs 参数高效微调	10
5.3 LoRA：低秩适配	11
5.4 LoRA 的实现	11
5.5 LoRA 的关键超参数	12
5.6 LoRA 的优势	12
5.7 本章小结	12
6 实用决策流程图	13
6.1 本章小结	13
7 总结与延伸	13
7.1 全课知识图谱	13
7.2 关键 Takeaways	14
7.3 拓展阅读	14

1 引言：高效训练神经网络的重要性

本节课的主题是高效神经网络训练 (Efficient Neural Network Training)，内容涵盖三大主题：(1) 混合精度训练，(2) 多 GPU 训练 (DDP 与 FSDP)，(3) 参数高效微调 (LoRA)。这些技术对于实际项目中的模型训练至关重要。

本课三大核心主题

1. **Mixed Precision Training** (混合精度训练)：用更少的比特表示参数和梯度，节省内存并加速计算
2. **Multi-GPU Training** (多 GPU 训练)：DDP 和 FSDP (ZeRO Stage 1-3) 的原理与实践
3. **Parameter Efficient Finetuning** (参数高效微调)：LoRA 的核心思想与实现

到目前为止，CS224N 课程已经讲解了如何将词转化为向量、如何将句子转化为向量、Transformer 架构、预训练等内容。本节课转向一个不同但同样重要的方向——如何在 GPU 上高效地训练大型模型。虽然这些内容“与自然语言没有直接关系”，但对于最终项目（和实际工作）极其有用。

1.1 本章小结

高效训练技术是将理论模型落地到实际 GPU 硬件上的桥梁。混合精度训练节省内存和时间，多 GPU 训练突破单卡容量限制，参数高效微调让资源受限的研究者也能适配大模型。

2 浮点数表示基础

2.1 FP32：标准单精度浮点数

在深度学习中，模型参数和梯度通常以浮点数的形式存储在 GPU 显存中。FP32 (32 位浮点数) 是最基本的数据类型，每个参数占用 4 字节。

FP32 的数值由以下公式计算：

$$(-1)^B \times 2^{E-127} \times \left(1 + \sum_{i=1}^{23} b_{23-i} \cdot 2^{-i} \right)$$

- B ：符号位 (sign)，决定正负
- E ：指数位 (exponent, 8 位)，决定**动态范围**——能表示多大或多小的数
- b_i ：尾数位 (mantissa, 23 位)，决定**精度**——数值的精确程度

指数位与尾数位的权衡

指数位越多，能表示的数的范围越大（更小的数和更大的数都能表示）。

尾数位越多，数值的精度越高（相邻可表示数之间的间隔更小）。

在总位数固定的前提下，指数位与尾数位之间存在权衡——这是理解 FP16、BF16 等格式的关键。

2.2 FP16：半精度浮点数

FP16 只有 16 位（2 字节），内存需求是 FP32 的一半。其格式为：1 位符号位 + 5 位指数位 + 10 位尾数位。

数据类型	总位数	指数位	尾数位
FP32	32	8	23
FP16	16	5	10
BF16	16	8	7

表 1: 三种常见浮点数格式的位数分配

FP16 的两个核心问题：

1. **动态范围不足**：指数位仅 5 位，导致非常小的数会被**下溢为零**（underflow）。例如，小于约 6×10^{-5} 的数在 FP16 中直接变为 0。
2. **精度不足**：尾数位仅 10 位，导致**舍入误差**。例如，1.0001 在 FP16 中会被舍入为 1.0。

FP16 梯度下溢是致命问题

在神经网络训练中，梯度通常非常小。如果直接用 FP16 计算梯度，**超过一半的梯度值会被四舍五入为零**，导致模型无法有效学习。这就是为什么不能简单地将所有计算切换到 FP16。

2.3 本章小结

浮点数的位数分配（指数位 vs 尾数位）决定了动态范围和精度之间的权衡。FP32 是标准格式（4 字节/参数），FP16 节省一半内存但存在严重的动态范围和精度问题。理解这些限制是混合精度训练的前提。

3 混合精度训练

3.1 初步方案：FP32 主权重 + FP16 计算

既然纯 FP16 有问题，一个自然的想法是：**保留一份 FP32 的模型副本（Master Weights），但在前向和反向传播中使用 FP16 计算。**

具体步骤：

1. 维护 FP32 格式的模型参数副本（Master Weights）
2. 将 Master Weights 转换为 FP16，执行前向传播
3. 在 FP16 中计算梯度（反向传播）
4. 将 FP16 梯度上转为 FP32
5. 用 FP32 梯度更新 Master Weights

⁰Slides 第 11 页。图来自 NVIDIA 博客。

6. 将更新后的 Master Weights 复制回 FP16 版本

梯度下溢问题仍未解决

虽然权重更新的精度问题通过 FP32 Master Weights 解决了，但反向传播中计算的**梯度本身仍然是 FP16 的**。那些极小的梯度值仍然会在 FP16 中下溢为零，丢失重要的学习信号。

3.2 Loss Scaling: 解决梯度下溢

为了避免 FP16 下的梯度下溢，引入 **Loss Scaling** 技巧：在计算梯度之前，将损失函数乘以一个较大的常数（如 1000），使得梯度也被等比放大。放大后的梯度值更大，不容易下溢为零。在 FP32 中更新权重之前，再将梯度除以该常数。

混合精度训练完整方案 (FP16 + Loss Scaling)

1. 维护 FP32 格式的 Master Weights
2. 在 FP16 中执行前向传播
3. 将损失乘以一个大常数 (Loss Scaling)，人为放大梯度
4. 在 FP16 中计算梯度（此时梯度已被放大，不容易下溢）
5. 将梯度转为 FP32 后除以 **Scaling Factor**
6. 用 FP32 更新 Master Weights
7. 复制回 FP16 版本

在 PyTorch 中，使用 `torch.cuda.amp.GradScaler` 和 `torch.cuda.amp.autocast` 可以方便地实现此方案：

```
1 scaler = torch.cuda.amp.GradScaler()
2
3 for input, target in data:
4     optimizer.zero_grad()
5     with torch.cuda.amp.autocast():
6         output = model(input)
7         loss = loss_fn(output, target)
8     scaler.scale(loss).backward()
9     scaler.step(optimizer)
10    scaler.update()
```

Listing 1: PyTorch 中的 FP16 混合精度训练（含 GradScaler）

Loss Scaling 的局限性

Loss Scaling 的常数需要精心选择：太小则梯度仍然下溢，太大则梯度上溢为 NaN。GradScaler 通过动态调整 Scaling Factor 来应对不同训练阶段的梯度分布变化，但这增加了训练的复杂性。

3.3 BFloat16: 更优雅的方案

Loss Scaling 的根本原因是 FP16 的动态范围不足（仅 5 位指数）。**BFloat16** (Brain Float 16, 由 Google Brain 提出) 提供了一个更根本的解决方案:

BF16 的设计理念:

- **8 位指数**: 与 FP32 完全相同的动态范围, 彻底消除下溢问题
- **7 位尾数**: 精度比 FP16 (10 位) 更低, 但实验表明对神经网络训练影响极小
- **无需 Loss Scaling**: 因为动态范围足够, 不再需要人为放大梯度

使用 BF16 的代码极其简洁:

```
1 # Creates model and optimizer in default precision
2 model = Net().cuda()
3 optimizer = optim.SGD(model.parameters(), ...)
4
5 for input, target in data:
6     optimizer.zero_grad()
7     # Enables autocasting for the forward pass
8     with torch.autocast(device_type="cuda"):
9         output = model(input)
10        loss = loss_fn(output, target)
11    # No GradScaler needed!
12    loss.backward()
13    optimizer.step()
```

Listing 2: PyTorch 中的 BF16 混合精度训练 (无需 GradScaler)

BF16 是当前最佳实践

如果你的 GPU 支持 BF16 (Ampere 架构及更新: A100、H100、A6000 等), **始终使用 BF16 而非 FP16**。BF16 无需 Loss Scaling, 代码更简洁, 训练更稳定。可通过 `torch.cuda.is_bf16_supported()` 检查支持情况。

3.4 混合精度训练的实际效果

以在单块 A100 上微调 DistilBERT 进行情感分类为例:

数据类型	训练时间	准确率	显存占用
Float64	~25 min	高	最大
Float32	基准	基准	基准
Mixed (BF16)	减少约 1/3	略有提升	显著减少

表 2: 不同精度下 DistilBERT 微调的性能对比

BF16 混合精度训练不仅节省了约 1/3 的训练时间和大量显存, 准确率甚至略有提升——这是因为低精度表示具有一定的**正则化效果**。训练加速的根本原因是: 半精度矩阵乘法在现代 GPU 的 Tensor Core 上速度更快。

3.5 本章小结

- FP16 节省内存但存在梯度下溢和精度问题，需要 Loss Scaling
- BF16 保留 FP32 的动态范围，牺牲精度，无需 Loss Scaling
- **混合精度训练应始终启用**：几乎不影响模型质量，显著节省时间和内存
- 新架构 GPU 优先使用 BF16，旧架构使用 FP16 + GradScaler

4 多 GPU 训练：DDP 与 ZeRO

4.1 GPU 显存占用分析

在讨论多 GPU 训练之前，需要理解单 GPU 上的显存都用在了哪里。以混合精度训练为例，每个参数的显存开销为：

组件	精度	每参数字节数
模型参数	FP16	2
梯度	FP16	2
Master Weights (FP32 副本)	FP32	4
Adam 动量 (momentum)	FP32	4
Adam 方差 (variance)	FP32	4
合计		16

表 3: 混合精度 + Adam 优化器的每参数显存开销

优化器状态的显存开销

很多人第一次了解到优化器也需要大量显存时会感到惊讶。使用 Adam 优化器时，每个参数需要额外存储动量 (momentum) 和方差 (variance)，各占 4 字节 (FP32)。加上 FP32 Master Weights，仅优化器状态就需要 **12 字节/参数**——是模型参数本身 (2 字节) 的 6 倍！

4.2 分布式数据并行 (DDP)

Distributed Data Parallel (DDP) 是最基本的多 GPU 训练策略。

DDP 的工作流程：

1. 将数据集均匀分割到 N 个 GPU 上 (每个 GPU 处理 $1/N$ 的数据)
2. 每个 GPU 维护**完整**的模型副本和优化器状态
3. 每个 GPU 独立执行前向传播和反向传播，得到各自的梯度
4. 通过 **All-Reduce** 操作同步梯度：汇总所有 GPU 的梯度并分发给所有 GPU
5. 每个 GPU 用汇总后的梯度更新自己的模型，保持同步

All-Reduce: DDP 的核心原语

All-Reduce 是一个 MPI（消息传递接口）原语，它将所有 GPU 上的数据进行归约（如求和），然后将结果广播到每个 GPU。在 DDP 中，All-Reduce 用于汇总梯度。通信开销为每参数 2 字节（FP16 梯度）。

4.3 DDP 的内存问题

DDP 虽然简单有效，但存在严重的**内存冗余**问题：

以一个 7.5B 参数的模型（ $\Psi = 7.5\text{B}$ ）为例，使用 Adam + 混合精度：

$$\text{每 GPU 显存} = (2 + 2 + K) \times \Psi = (2 + 2 + 12) \times 7.5\text{B} = 120\text{GB}$$

其中 $K = 12$ （FP32 Master Weights 4B + Adam momentum 4B + Adam variance 4B）。A100 有 80GB 显存，**连一块卡都放不下**。而且，每个 GPU 上的优化器状态完全相同——这是巨大的浪费。

DDP 无法解决大模型问题

DDP 不减少任何单 GPU 的显存占用——它只是通过数据并行来加速训练。如果单个 GPU 放不下模型 + 优化器状态，DDP 无能为力。我们需要更聪明的策略来**分摊**显存开销。

4.4 ZeRO Stage 1: 分片优化器状态

ZeRO (Zero Redundancy Optimizer) 由微软 DeepSpeed 项目提出，核心思想是将冗余的状态**分片 (shard)** 存储到不同 GPU 上。

ZeRO Stage 1 只分片**优化器状态**（绿色部分）：

Stage 1 的工作流程：

1. 每个 GPU 持有完整的模型参数（FP16）和完整的梯度
2. 但只持有 $1/N$ 的优化器状态
3. 反向传播后，通过 **Reduce-Scatter** 操作，每个 GPU 只接收自己负责的参数分片的汇总梯度
4. 每个 GPU 用本地优化器状态更新自己负责的参数分片
5. 通过 **All-Gather** 操作，收集所有 GPU 更新后的参数，恢复完整模型

ZeRO Stage 1 是免费的午餐

关键洞察：**All-Reduce = Reduce-Scatter + All-Gather**。DDP 需要执行一次 All-Reduce，而 ZeRO Stage 1 执行 Reduce-Scatter + All-Gather，通信量完全相同。因此，ZeRO Stage 1 在**不增加任何通信开销**的情况下节省了优化器状态的显存——你应该始终使用它。

三个 MPI 原语

- **All-Reduce**: 所有 GPU 的数据归约后广播到所有 GPU (用于 DDP)
- **Reduce-Scatter**: 所有 GPU 的数据归约后, 结果的不同分片散发到不同 GPU
- **All-Gather**: 收集所有 GPU 的分片, 拼接成完整数据广播到所有 GPU

核心恒等式: $\text{All-Reduce} \equiv \text{Reduce-Scatter} + \text{All-Gather}$

4.5 ZeRO Stage 2: 分片梯度

Stage 2 在 Stage 1 的基础上, 进一步分片**梯度**。

核心技巧: **永远不实例化完整的梯度向量**。反向传播是逐层进行的——当计算完第 j 层的梯度后, 立即通过 Reduce 操作将该层梯度发送给负责该层的 GPU, 然后释放本地的临时梯度内存。

Stage 2 的通信量也与 DDP 相当 ($\text{Reduce} + \text{All-Gather} \approx \text{All-Reduce}$), 因此也是**基本免费的**显存优化。

ZeRO Stage 1 和 2 应始终启用

ZeRO Stage 1 (分片优化器状态) 和 Stage 2 (分片梯度) 都不增加显著的通信开销, 却能大幅减少每 GPU 的显存占用。在多 GPU 训练中, **没有理由不使用它们**。

4.6 ZeRO Stage 3 / FSDP: 分片模型参数

当模型大到连参数本身都放不进单个 GPU 时, 就需要 **ZeRO Stage 3**, 也称为 **FSDP (Fully Sharded Data Parallel)**。

Stage 3 连模型参数也进行分片:

1. 将模型划分为多个 **FSDP 单元**
2. 将每个单元的参数展平 (Flat Parameter), 分配到不同 GPU
3. **前向传播时**: 在计算某层之前, 通过 All-Gather 临时收集该层的完整参数; 计算完成后丢弃非本地分片
4. **反向传播时**: 再次 All-Gather 获取完整参数, 计算梯度后通过 Reduce-Scatter 分发梯度
5. 每个 GPU 用本地优化器状态更新本地参数分片

Stage 3 有额外的通信开销

与 Stage 1/2 不同, Stage 3 需要在前向传播和反向传播中都执行 All-Gather (分别收集参数), 加上反向传播中的 Reduce-Scatter。总通信量为 **2 次 All-Gather + 1 次 Reduce-Scatter**, 显著高于 DDP。但如果模型放不进单个 GPU, 这是唯一的选择。

好消息是: 当模型足够大时, 前向传播一层的计算时间足以**预取** (prefetch) 下一层的参数。PyTorch 的 FSDP 默认启用这种计算-通信重叠, 使得通信开销在实际中被大幅隐藏。

4.7 GPU 显存中被忽视的部分：模型激活值

在前面的分析中，我们忽略了一个重要的显存占用来源：**模型激活值** (Activations)。

反向传播需要保存前向传播中每一层的激活值，其显存占用与 **batch size** **线性相关**。这解释了为什么增大 batch size 时容易遇到 OOM 错误。ZeRO 的三个阶段都不处理激活值分片——要解决激活值的显存问题，需要使用 **Activation Checkpointing** (梯度检查点) 技术。

Activation Checkpointing

Activation Checkpointing (也叫 Gradient Checkpointing) 的核心思想：不保存所有层的激活值，只保存部分“检查点”层的激活值。反向传播到某层时，从最近的检查点重新计算中间激活值。这是用**额外的计算时间**换取**更少的显存占用**。

4.8 本章小结

- **DDP**：数据并行，每个 GPU 维护完整的模型/梯度/优化器状态，通过 All-Reduce 同步梯度
- **ZeRO Stage 1**：分片优化器状态，通信量与 DDP 相同（免费优化）
- **ZeRO Stage 2**：分片优化器状态 + 梯度，通信量与 DDP 相当（基本免费）
- **ZeRO Stage 3 / FSDP**：分片一切（参数 + 梯度 + 优化器状态），有额外通信开销，但支持超大模型
- 模型激活值的显存与 batch size 线性相关，需要 Activation Checkpointing 另行处理

5 参数高效微调：LoRA

5.1 为什么需要参数高效微调

当以下所有手段都无法解决 OOM 问题时——混合精度训练、ZeRO Stage 3、Activation Checkpointing、batch size 已经为 1——就需要考虑**参数高效微调** (Parameter-Efficient Fine-Tuning, PEFT)。

参数高效微调的动机不仅仅是显存限制：

参数高效微调的三大动机

1. **显存限制**：GPT-3 有 1750 亿参数，全量微调需要海量显存
2. **环境与成本**：训练 GPT-3 的碳排放相当于运行燃煤电厂 10 小时；AI 计算需求即将超过全球计算产能
3. **科学动机**：大模型严重过参数化，对于小数据集的下游任务，参数高效微调可能泛化更好（起到正则化作用）

5.2 全量微调 vs 参数高效微调

全量微调 (Full Fine-Tuning)：更新模型的所有参数 θ ，寻找最优的 $\Delta\theta$ 。

$$\theta_{\text{new}} = \theta_{\text{pretrained}} + \Delta\theta, \quad |\Delta\theta| = |\theta|$$

参数高效微调：只更新一小部分参数，搜索空间远小于全量微调。

$$\theta_{\text{new}} = \theta_{\text{pretrained}} + \Delta\theta, \quad |\Delta\theta| \ll |\theta|$$

参数高效微调的额外好处：每个任务只需存储一个很小的 $\Delta\theta$ （而非完整的模型副本），在多任务场景下极其高效。

5.3 LoRA：低秩适配

LoRA (Low-Rank Adaptation) 是当前最流行的参数高效微调方法。它基于一个经验观察：大模型微调时的梯度具有低内在秩 (low intrinsic rank)。

对于一个预训练权重矩阵 $W \in \mathbb{R}^{d \times k}$ ，LoRA 将更新约束为低秩形式：

$$W_{\text{new}} = W + \alpha \cdot B \cdot A$$

- $W \in \mathbb{R}^{d \times k}$ ：预训练权重（冻结，不参与训练）
- $A \in \mathbb{R}^{r \times k}$ ：LoRA 矩阵 A（可训练）
- $B \in \mathbb{R}^{d \times r}$ ：LoRA 矩阵 B（可训练）
- r ：秩，远小于 d 和 k （典型值： $r = 8$ ）
- α ：缩放系数，控制预训练知识与新任务知识的权衡

LoRA 的参数节省

原始权重矩阵 W 有 $d \times k$ 个参数。LoRA 只需学习 A ($r \times k$ 个参数) 和 B ($d \times r$ 个参数)，共 $r \times (d + k)$ 个参数。当 $r \ll \min(d, k)$ 时，可训练参数量远小于原始参数量。例如， $d = k = 768$ ， $r = 8$ 时，可训练参数量仅为原来的 $\frac{2 \times 8}{768} \approx 2\%$ 。

5.4 LoRA 的实现

在代码层面，LoRA 的实现非常简洁：

```
1 input_dim = 768    # hidden size of pre-trained model
2 output_dim = 768   # output size of the layer
3 rank = 8           # rank 'r' for low-rank adaptation
4
5 W = ...           # from pretrained network, shape: input_dim x output_dim
6
7 W_A = nn.Parameter(torch.empty(input_dim, rank)) # LoRA weight A
8 W_B = nn.Parameter(torch.empty(rank, output_dim)) # LoRA weight B
9
10 # Initialization
11 nn.init.kaiming_uniform_(W_A, a=math.sqrt(5))
12 nn.init.zeros_(W_B) # B initialized to zero!
13
14 def regular_forward_matmul(x, W):
15     h = x @ W
16     return h
```

```

17
18 def lora_forward_matmul(x, W, W_A, W_B):
19     h = x @ W          # regular matrix multiplication (frozen)
20     h += x @ (W_A @ W_B) * alpha # add scaled LoRA weights
21     return h

```

Listing 3: LoRA 的 PyTorch 实现

LoRA 初始化策略

B 矩阵初始化为全零， A 矩阵使用 Kaiming 初始化。这意味着训练开始时 $\Delta W = B \cdot A = 0$ ，模型行为与预训练模型完全一致。随着训练进行， B 逐渐偏离零，模型开始适配下游任务。这种初始化确保了训练的稳定性。

5.5 LoRA 的关键超参数

- 应用位置**：LoRA 应用于 Self-Attention 中的 **Query 矩阵** (W_q) 和 **Value 矩阵** (W_v)，这是经验上效果最好的选择。
- 秩 r** ：实验表明，即使 r 非常小 ($r = 1$ 或 $r = 4$)，LoRA 也能获得很好的性能。 $r = 8$ 是一个推荐的起始值。
- 缩放系数 α** ：通常设为 1。 $\alpha < 1$ 表示更偏向保留预训练知识， $\alpha > 1$ 表示更偏向学习新任务知识。

LoRA 实用指南

- 应用于 **Query 和 Value 矩阵**
- 秩设为 $r = 8$
- α 设为 1
- 这些默认值在大多数场景下效果很好，可以作为起点

5.6 LoRA 的优势

- 显存高效**：只需存储和优化两个小矩阵 A 和 B ，大幅减少梯度和优化器状态的显存
- 推理无开销**：训练完成后，可以将 $\alpha \cdot B \cdot A$ 直接合并到 W 中，推理时与原模型无异
- 多任务高效**：每个任务只需存储一组小的 A, B 矩阵；切换任务时只需替换 LoRA 权重
- 收敛于全量微调**：当 r 增大到等于 $\min(d, k)$ 时，LoRA 等价于全量微调——提供了一个连续的控制旋钮
- 正则化效果**：低秩约束天然地限制了搜索空间，在小数据集上可能泛化更好

5.7 本章小结

- LoRA 将权重更新约束为低秩矩阵乘积 $\Delta W = B \cdot A$ ，大幅减少可训练参数
- 应用于 Self-Attention 的 W_q 和 W_v 矩阵，秩 $r = 8$ ， $\alpha = 1$ 是好的起点

- 推理时无额外开销，多任务时只需存储小矩阵
- 在资源受限环境下，LoRA 是微调大模型的首选方案

6 实用决策流程图

Shikhar Murty 在课程结尾给出了一个非常实用的决策流程图，帮助你在实际项目中选择合适的训练策略：

高效训练决策流程

1. **始终使用混合精度训练**。如果 GPU 支持 Ampere 架构 (A100/H100/A6000)，使用 BF16；否则使用 FP16 + GradScaler
2. 尝试 **batch size = 1**。如果能放下：
 - 增大 batch size
 - 始终使用 **ZeRO Stage 2** (免费优化)
3. 如果 batch size = 1 也放不下：
 - 尝试 **ZeRO Stage 3 / FSDP**
 - 尝试 **Activation Checkpointing**
4. 如果以上全部失败：使用 **LoRA**
 - 应用于 W_q 和 W_v
 - 秩 $r = 8$, $\alpha = 1$

此流程假设多 GPU 环境

上述流程中的 ZeRO / FSDP 假设你有多个 GPU。如果只有单个 GPU，ZeRO 无法帮助你——此时应直接考虑量化 (quantization) 或 LoRA 等技术。

6.1 本章小结

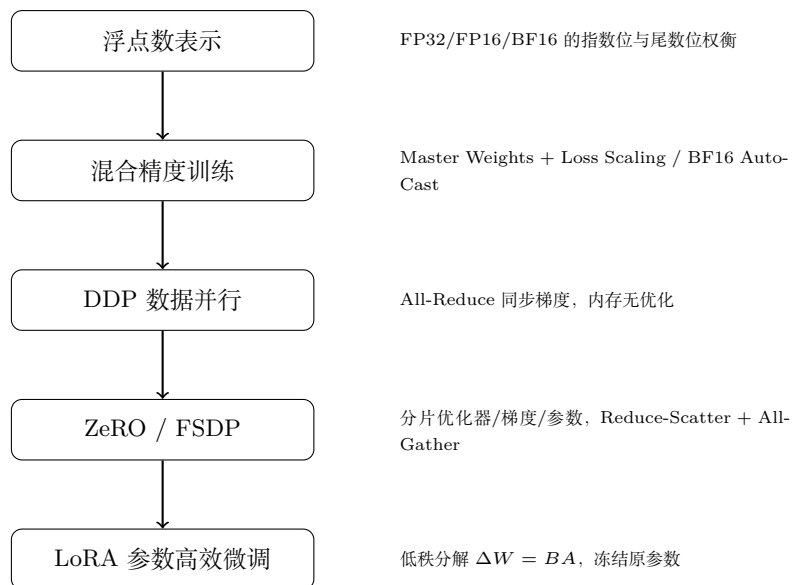
决策流程的核心逻辑是“从简单到复杂”：先用零成本的优化（混合精度、ZeRO Stage 1/2），再考虑有代价的优化（ZeRO Stage 3），最后采用改变训练范式的方法（LoRA）。在每一步都优先选择开销最小的方案。

7 总结与延伸

7.1 全课知识图谱

本课建立了一条从数值表示到训练策略的完整认知链：

⁰Slides 第 65 页 (最后一页)。



7.2 关键 Takeaways

五条核心原则

1. **始终启用混合精度训练**: BF16 (Ampere+) 或 FP16 + GradScaler, 几乎没有质量损失
2. **优化器状态是显存大户**: Adam 的 momentum 和 variance 占 12 字节/参数, 远超模型本身
3. **ZeRO Stage 1/2 免费**: 利用 All-Reduce = Reduce-Scatter + All-Gather 的等价性, 不增加通信即可节省显存
4. **FSDP 支持超大模型**: 有额外通信开销, 但可通过计算-通信重叠来隐藏
5. **LoRA 是最后的利器**: 当全量微调不可行时, 低秩适配以极少的参数达到接近全量微调的效果

7.3 拓展阅读

- Micikevicius et al., 2018. *Mixed Precision Training*. ICLR 2018. <https://arxiv.org/abs/1710.03740>
- Rajbhandari et al., 2020. *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*. SC 2020. <https://arxiv.org/abs/1910.02054>
- Hu et al., 2021. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR 2022. <https://arxiv.org/abs/2106.09685>
- PyTorch FSDP 文档: <https://pytorch.org/docs/stable/fsdp.html>
- PyTorch AMP 文档: <https://pytorch.org/docs/stable/amp.html>
- Hugging Face PEFT 库: <https://github.com/huggingface/peft>
- DeepSpeed ZeRO 文档: <https://www.deepspeed.ai/tutorials/zero/>